Business case: Delhivery

Problem Statement

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. The company wants to understand and process the data coming out of data engineering pipelines.Clean, sanitize and manipulate data to get useful features out of raw fields.Make sense out of the raw data and help the data science team to build forecasting models on it.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('delhivery_data.csv')
```

```
df.shape
```

```
(144867, 24)
```

```
df.isna().sum()
```

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	293
destination_center	0
destination_name	261
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

```
df.isna().sum() /len(df) * 100
```

	0
data	0.000000
trip_creation_time	0.000000
route_schedule_uuid	0.000000
route_type	0.000000
trip_uuid	0.000000
source_center	0.000000
source_name	0.202254
destination_center	0.000000
destination_name	0.180165
od_start_time	0.000000
od_end_time	0.000000
start_scan_to_end_scan	0.000000
is_cutoff	0.000000
cutoff_factor	0.000000
cutoff_timestamp	0.000000
actual_distance_to_destination	0.000000
actual_time	0.000000
osrm_time	0.000000
osrm_distance	0.000000
factor	0.000000
segment_actual_time	0.000000
segment_osrm_time	0.000000
segment_osrm_distance	0.000000
segment_factor	0.000000

dtype: float64

```
df.dropna(inplace=True)
```

```
df.isna().sum()
```

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	0
destination_center	0
destination_name	0
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

```
df[df.duplicated()]
```

data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destina

0 rows × 24 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 144316 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                         Non-Null Count   Dtype
---  ------                         --------------   -----
 0   data                           144316 non-null  object
 1   trip_creation_time             144316 non-null  object
 2   route_schedule_uuid            144316 non-null  object
 3   route_type                     144316 non-null  object
 4   trip_uuid                      144316 non-null  object
 5   source_center                  144316 non-null  object
 6   source_name                    144316 non-null  object
 7   destination_center             144316 non-null  object
 8   destination_name               144316 non-null  object
 9   od_start_time                  144316 non-null  object
 10  od_end_time                    144316 non-null  object
 11  start_scan_to_end_scan         144316 non-null  float64
 12  is_cutoff                      144316 non-null  bool
 13  cutoff_factor                  144316 non-null  int64
 14  cutoff_timestamp               144316 non-null  object
 15  actual_distance_to_destination 144316 non-null  float64
 16  actual_time                    144316 non-null  float64
 17  osrm_time                      144316 non-null  float64
 18  osrm_distance                  144316 non-null  float64
 19  factor                         144316 non-null  float64
 20  segment_actual_time            144316 non-null  float64
 21  segment_osrm_time              144316 non-null  float64
 22  segment_osrm_distance          144316 non-null  float64
 23  segment_factor                 144316 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
```

```
memory usage: 26.6+ MB
```

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
start_scan_to_end_scan	144316.0	963.697698	1038.082976	20.000000	161.000000	451.000000	1645.000000	7898.000000
cutoff_factor	144316.0	233.561345	345.245823	9.000000	22.000000	66.000000	286.000000	1927.000000
actual_distance_to_destination	144316.0	234.708498	345.480571	9.000045	23.352027	66.135322	286.919294	1927.447705
actual_time	144316.0	417.996237	598.940065	9.000000	51.000000	132.000000	516.000000	4532.000000
osrm_time	144316.0	214.437055	308.448543	6.000000	27.000000	64.000000	259.000000	1686.000000
osrm_distance	144316.0	285.549785	421.717826	9.008200	29.896250	78.624400	346.305400	2326.199100
factor	144316.0	2.120178	1.717065	0.144000	1.604545	1.857143	2.212280	77.387097
segment_actual_time	144316.0	36.175379	53.524298	-244.000000	20.000000	28.000000	40.000000	3051.000000
segment_osrm_time	144316.0	18.495697	14.774008	0.000000	11.000000	17.000000	22.000000	1611.000000
segment_osrm_distance	144316.0	22.818993	17.866367	0.000000	12.053975	23.508300	27.813325	2191.403700
segment_factor	144316.0	2.218707	4.854804	-23.444444	1.347826	1.684211	2.250000	574.250000

```
df.describe(include=object).T
```

	count	unique	top	freq
data	144316	2	training	104632
trip_creation_time	144316	14787	2018-09-25 04:21:12.551117	101
route_schedule_uuid	144316	1497	thanos::sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...	1812
route_type	144316	2	FTL	99132
trip_uuid	144316	14787	trip-153784927255069118	101
source_center	144316	1496	IND000000ACB	23267
source_name	144316	1496	Gurgaon_Bilaspur_HB (Haryana)	23267
destination_center	144316	1466	IND000000ACB	15192
destination_name	144316	1466	Gurgaon_Bilaspur_HB (Haryana)	15192
od_start_time	144316	26223	2018-09-21 18:37:09.322207	81
od_end_time	144316	26223	2018-09-24 09:59:15.691618	81
cutoff_timestamp	144316	92894	2018-09-24 05:19:20	39

```
df['data'] = df['data'].astype('category')
df['route_type'] = df['route_type'].astype('category')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 144316 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   data                          144316 non-null  category
 1   trip_creation_time            144316 non-null  object
 2   route_schedule_uuid           144316 non-null  object
 3   route_type                    144316 non-null  category
 4   trip_uuid                     144316 non-null  object
 5   source_center                 144316 non-null  object
 6   source_name                   144316 non-null  object
 7   destination_center            144316 non-null  object
 8   destination_name              144316 non-null  object
 9   od_start_time                 144316 non-null  object
 10  od_end_time                   144316 non-null  object
 11  start_scan_to_end_scan        144316 non-null  float64
 12  is_cutoff                     144316 non-null  bool
 13  cutoff_factor                 144316 non-null  int64
 14  cutoff_timestamp              144316 non-null  object
 15  actual_distance_to_destination 144316 non-null  float64
 16  actual_time                   144316 non-null  float64
 17  osrm_time                     144316 non-null  float64
 18  osrm_distance                 144316 non-null  float64
 19  factor                        144316 non-null  float64
 20  segment_actual_time           144316 non-null  float64
 21  segment_osrm_time             144316 non-null  float64
 22  segment_osrm_distance         144316 non-null  float64
```

```
 23  segment_factor                  144316 non-null  float64
dtypes: bool(1), category(2), float64(10), int64(1), object(10)
memory usage: 24.6+ MB
```

```
# is_cutoff, cutoff_factor, cutoff_timestamp, factor, segment_factor are given as unknown fields, therefore droping them.

unknown_fields = df[['is_cutoff', 'cutoff_factor', 'cutoff_timestamp', 'factor', 'segment_factor']]
unknown_fields
```

	is_cutoff	cutoff_factor	cutoff_timestamp	factor	segment_factor
0	True	9	2018-09-20 04:27:55	1.272727	1.272727
1	True	18	2018-09-20 04:17:55	1.200000	1.111111
2	True	27	2018-09-20 04:01:19.505586	1.428571	2.285714
3	True	36	2018-09-20 03:39:57	1.550000	1.750000
4	False	39	2018-09-20 03:33:55	1.545455	1.200000
...	...	...	...	...	...
144862	True	45	2018-09-20 21:57:20	1.566667	1.000000
144863	True	54	2018-09-20 21:31:18	1.578947	1.238095
144864	True	63	2018-09-20 21:11:18	1.590909	0.588235
144865	True	72	2018-09-20 20:53:19	1.612245	0.629630
144866	False	70	2018-09-20 16:24:28.436231	4.484211	29.777778

144316 rows × 5 columns

```
df.drop(unknown_fields, axis=1, inplace=True)
```

```
df.head()
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destina
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IN
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IN
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IN
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IN
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IN

```
df.columns
```

```
Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type',
       'trip_uuid', 'source_center', 'source_name', 'destination_center',
       'destination_name', 'od_start_time', 'od_end_time',
       'start_scan_to_end_scan', 'actual_distance_to_destination',
       'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time',
       'segment_osrm_time', 'segment_osrm_distance'],
      dtype='object')
```

```
df[['trip_creation_time','od_start_time','od_end_time']]

#these columns contain time related data therefore need to convert them to datetime format
```

	trip_creation_time	od_start_time	od_end_time
0	2018-09-20 02:35:36.476840	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797
1	2018-09-20 02:35:36.476840	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797
2	2018-09-20 02:35:36.476840	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797
3	2018-09-20 02:35:36.476840	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797
4	2018-09-20 02:35:36.476840	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797
...	...	...	...
144862	2018-09-20 16:24:28.436231	2018-09-20 16:24:28.436231	2018-09-20 23:32:09.618069
144863	2018-09-20 16:24:28.436231	2018-09-20 16:24:28.436231	2018-09-20 23:32:09.618069
144864	2018-09-20 16:24:28.436231	2018-09-20 16:24:28.436231	2018-09-20 23:32:09.618069
144865	2018-09-20 16:24:28.436231	2018-09-20 16:24:28.436231	2018-09-20 23:32:09.618069
144866	2018-09-20 16:24:28.436231	2018-09-20 16:24:28.436231	2018-09-20 23:32:09.618069

144316 rows × 3 columns

```
df['trip_creation_time'] = pd.to_datetime(df['trip_creation_time'])
df['od_start_time'] = pd.to_datetime(df['od_start_time'])
df['od_end_time'] = pd.to_datetime(df['od_end_time'])
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 144316 entries, 0 to 144866
Data columns (total 19 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144316 non-null  category
 1   trip_creation_time              144316 non-null  datetime64[ns]
 2   route_schedule_uuid             144316 non-null  object
 3   route_type                      144316 non-null  category
 4   trip_uuid                       144316 non-null  object
 5   source_center                   144316 non-null  object
 6   source_name                     144316 non-null  object
 7   destination_center              144316 non-null  object
 8   destination_name                144316 non-null  object
 9   od_start_time                   144316 non-null  datetime64[ns]
 10  od_end_time                     144316 non-null  datetime64[ns]
 11  start_scan_to_end_scan          144316 non-null  float64
 12  actual_distance_to_destination  144316 non-null  float64
 13  actual_time                     144316 non-null  float64
 14  osrm_time                       144316 non-null  float64
 15  osrm_distance                   144316 non-null  float64
 16  segment_actual_time             144316 non-null  float64
 17  segment_osrm_time               144316 non-null  float64
 18  segment_osrm_distance           144316 non-null  float64
dtypes: category(2), datetime64[ns](3), float64(8), object(6)
memory usage: 20.1+ MB
```

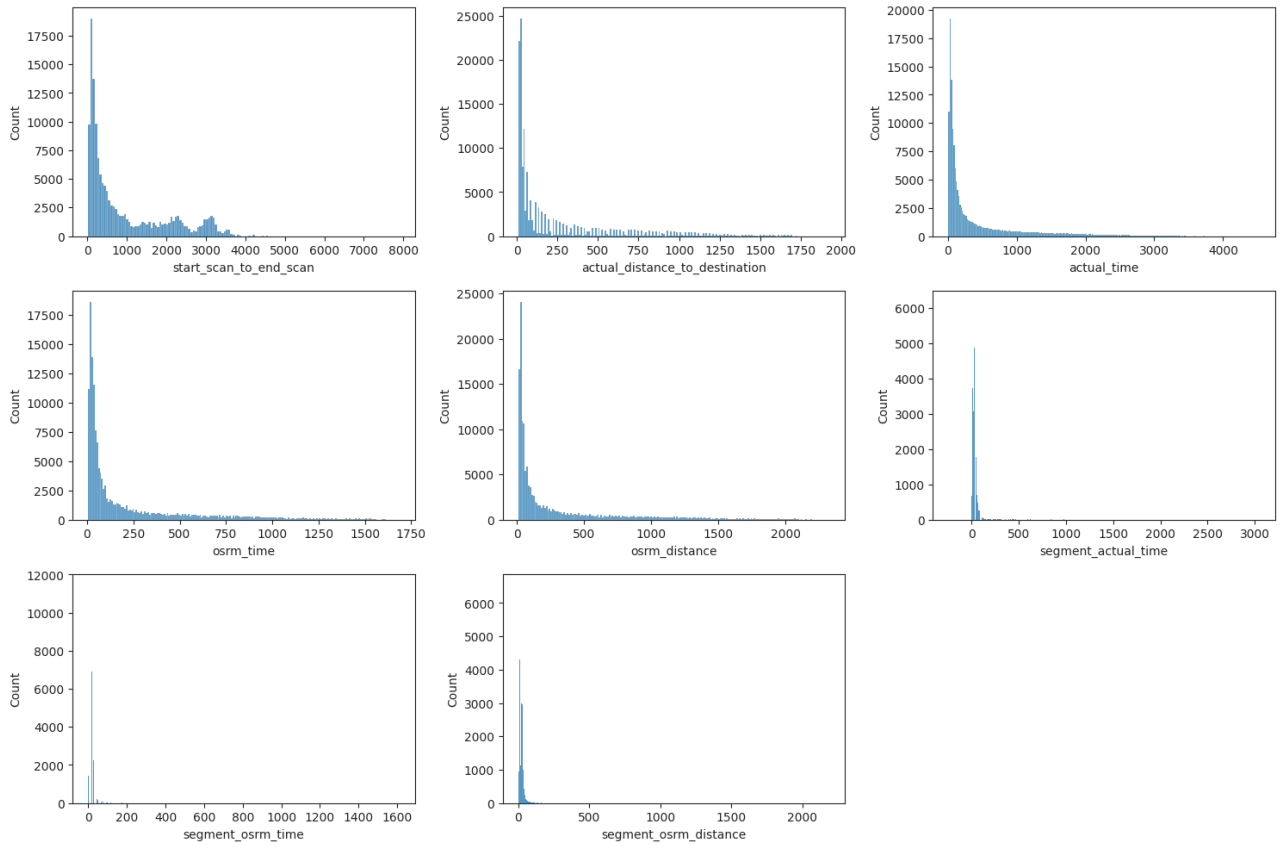
Visual Analysis

1) Distributions of all continous variables

```
columns_objects = df.select_dtypes(include=['number'])
columns_objects
```

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	seg
0	86.0	10.435660	14.0	11.0	11.9653	14.0	

```
plt.figure(figsize=(15, 10)) # Adjust the figure size as needed

for i, col in enumerate(columns_objects):
  plt.subplot(3, 3, i + 1)
  sns.histplot(df[col])

plt.tight_layout() # Adjust layout to prevent overlapping titles and labels
plt.show()
```



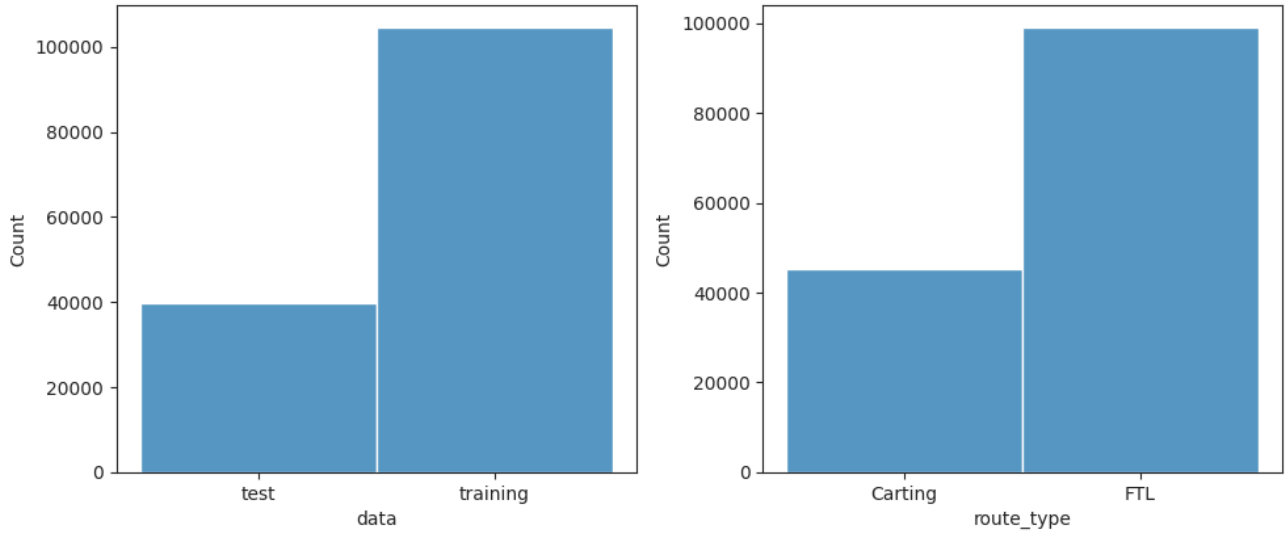
2) Distributions of categorical variables

```
columns_cat = df.select_dtypes(include=['category'])
columns_cat
```

	data	route_type
0	training	Carting
1	training	Carting
2	training	Carting
3	training	Carting
4	training	Carting
...	...	...
144862	training	Carting
144863	training	Carting
144864	training	Carting
144865	training	Carting
144866	training	Carting

144316 rows × 2 columns

```
plt.figure(figsize=(10,8))

for i, col in enumerate(columns_cat):
  plt.subplot(2,2, i+1)
  sns.histplot(df[col])

plt.tight_layout() # Adjust layout to prevent overlapping titles and labels
plt.show()
```

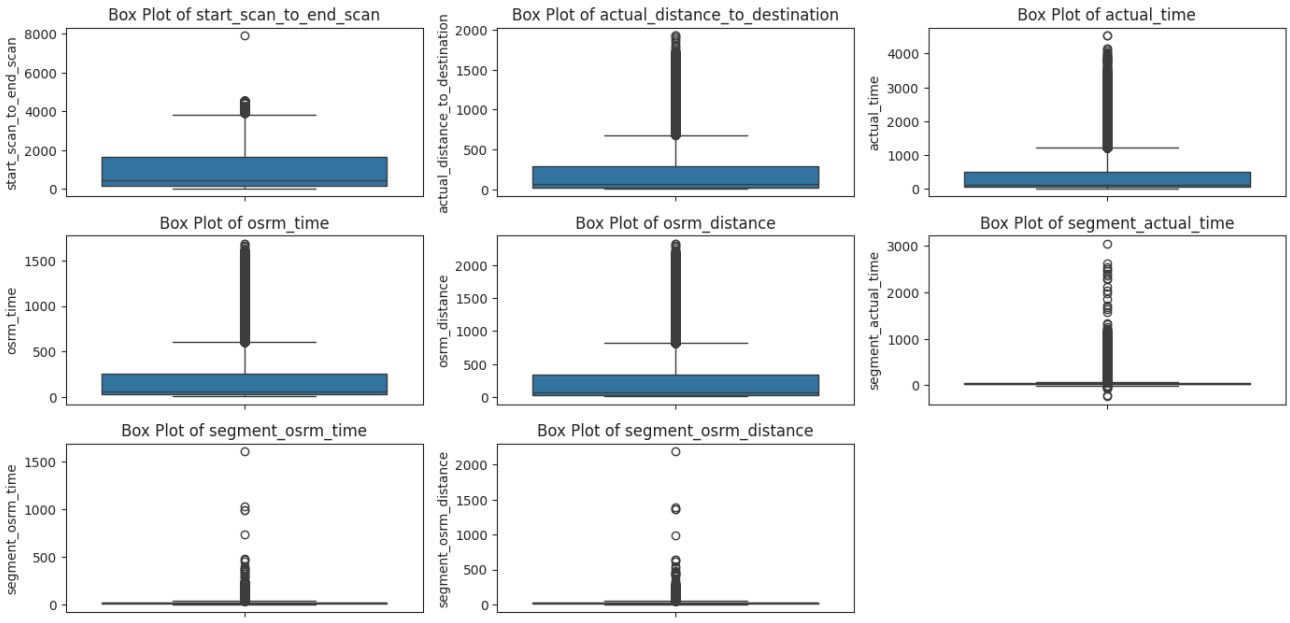


3) Distributions of all numerical variables

```
columns_num = df.select_dtypes(include='number')
columns_num
```

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	seg
0	86.0	10.435660	14.0	11.0	11.0653	14.0	

```
plt.figure(figsize=(14,9))

for i, col in enumerate(columns_num):
  plt.subplot(4,3,i+1)
  sns.boxplot(df[col])
  plt.title(f'Box Plot of {col}') # Add title to each subplot
  plt.xticks(rotation=45, ha='right') # Rotate x-axis labels

plt.tight_layout()
plt.show()
```




```
df['source_city'] = df["source_name"].str.extract(r'([^_]+)')
# source city
df['source_place'] = df["source_name"].str.split("_", n = 2, expand = True)[1]
# source place
df['source_code'] = (df["source_name"].str.split("_",n=2,expand = True)[2]).str.split(" ",n = 1, expand = True)[0]
# source code
df['source_state'] = df["source_name"].str.extract(r'.*\((.*)\).*')
#  source state
df['destination_city']= df["destination_name"].str.extract(r'([^_]+)')
# destination city
df['destination_place'] = df["destination_name"].str.split("_", n = 2, expand = True)[1]
# destination place
df['destination_code'] = (df["destination_name"].str.split("_",n=2,expand = True)[2]).str.split(" ",n = 1, expand = True)[0]
# destination code
df['destination_state'] = df["destination_name"].str.extract(r'.*\((.*)\).*')
# destination state
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 144316 entries, 0 to 144866
Data columns (total 27 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144316 non-null  category
 1   trip_creation_time              144316 non-null  datetime64[ns]
 2   route_schedule_uuid             144316 non-null  object
 3   route_type                      144316 non-null  category
 4   trip_uuid                       144316 non-null  object
 5   source_center                   144316 non-null  object
 6   source_name                     144316 non-null  object
```

```
 7   destination_center              144316 non-null  object
 8   destination_name                144316 non-null  object
 9   od_start_time                   144316 non-null  datetime64[ns]
 10  od_end_time                     144316 non-null  datetime64[ns]
 11  start_scan_to_end_scan          144316 non-null  float64
 12  actual_distance_to_destination  144316 non-null  float64
 13  actual_time                     144316 non-null  float64
 14  osrm_time                       144316 non-null  float64
 15  osrm_distance                   144316 non-null  float64
 16  segment_actual_time             144316 non-null  float64
 17  segment_osrm_time               144316 non-null  float64
 18  segment_osrm_distance           144316 non-null  float64
 19  source_city                     144316 non-null  object
 20  source_place                    142218 non-null  object
 21  source_code                     129688 non-null  object
 22  source_state                    144316 non-null  object
 23  destination_city                144316 non-null  object
 24  destination_place               141884 non-null  object
 25  destination_code                128883 non-null  object
 26  destination_state               144316 non-null  object
dtypes: category(2), datetime64[ns](3), float64(8), object(14)
memory usage: 28.9+ MB
```

```
# creating a new column to save the time taken from order taken to order received
df['od_duration_in_hrs'] = (df['od_end_time'] - df['od_start_time']).dt.total_seconds() / 3600
# converting minutes into hours
df['start_scan_to_end_scan'] = df['start_scan_to_end_scan']/60
# Trip_creation_time: Extract features like month, year and day.
df['trip_creation_month'] = df['trip_creation_time'].dt.month_name()
df['trip_creation_year'] = df['trip_creation_time'].dt.year
```

```
agg_dict_intermediate_dest = {
'data' : 'first',
'trip_creation_time' : 'first',
'route_type' : 'first',
'route_schedule_uuid' : 'first',
'source_center' : 'first',
'source_code' : 'first',
'source_city' : 'first',
'source_state' : 'first',
'source_name' : 'first',
'destination_center' : 'last',
'destination_code' : 'last',
'destination_city' : 'last',
'destination_state' : 'last',
'destination_name' : 'last',
'actual_distance_to_destination' : 'last',
'trip_creation_month':'first',
'trip_creation_year':'first'
}
```

```
df1 = df.groupby('trip_uuid').agg(agg_dict_intermediate_dest).reset_index()
```

```
df1.shape
```

```
(14787, 18)
```

```
df_part1 = pd.DataFrame(df.groupby('trip_uuid')['start_scan_to_end_scan'].unique().apply(sum).reset_index())

df_part2 = pd.DataFrame(df.groupby('trip_uuid')['od_duration_in_hrs'].unique().apply(sum).reset_index())

df_part3 =pd.DataFrame(df.groupby(['trip_uuid','od_duration_in_hrs'])['actual_time'].max().reset_index())
df_part3 = pd.DataFrame(df_part3.groupby('trip_uuid')['actual_time'].sum().reset_index())

df_part4 =pd.DataFrame(df.groupby(['trip_uuid','od_duration_in_hrs'])['osrm_time'].max().reset_index())
df_part4 = pd.DataFrame(df_part4.groupby('trip_uuid')['osrm_time'].sum().reset_index())

# actual time and segment actual time
seg_time = pd.DataFrame(df.groupby(['trip_uuid','od_duration_in_hrs'])['segment_actual_time'].sum().reset_index())
df_part5 = pd.DataFrame(seg_time.groupby('trip_uuid')['segment_actual_time'].sum().reset_index())

# osrm distance and segment osrm distance
om_dist =pd.DataFrame(df.groupby(['trip_uuid','od_duration_in_hrs'])['osrm_distance'].max().reset_index())
df_part6= pd.DataFrame(om_dist.groupby('trip_uuid')['osrm_distance'].sum().reset_index())
df_part7 = pd.DataFrame(df.groupby('trip_uuid')['segment_osrm_distance'].sum().reset_index())

# osrm time ans segment osrm time
df_part8 = pd.DataFrame(df.groupby('trip_uuid')['segment_osrm_time'].sum().reset_index())
```

```
df1 = df1.merge(df_part1,on='trip_uuid')
df1 = df1.merge(df_part2,on='trip_uuid')
```

```
df1 = df1.merge(df_part2,on='trip_uuid')
df1 = df1.merge(df_part3,on='trip_uuid')
df1 = df1.merge(df_part4,on='trip_uuid')
df1 = df1.merge(df_part5,on='trip_uuid')
df1 = df1.merge(df_part6,on='trip_uuid')
df1 = df1.merge(df_part7,on='trip_uuid')
df1 = df1.merge(df_part8,on='trip_uuid')
```

```
df1.shape
```

```
(14787, 26)
```

```
df.shape
```

```
(144316, 30)
```

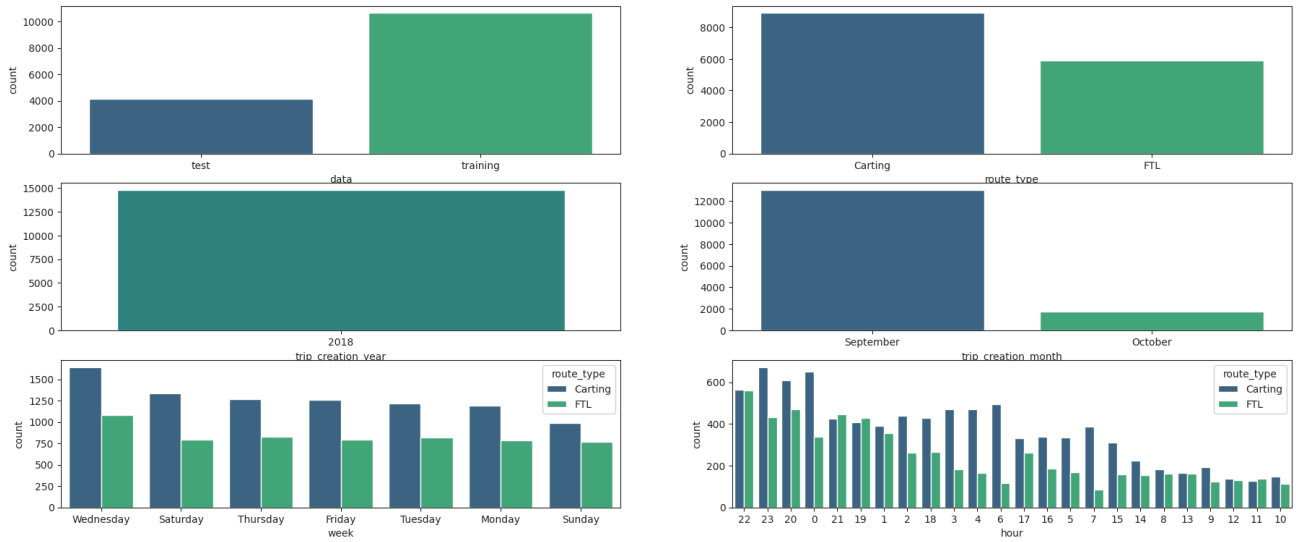
```
df = df1.copy(deep=True)
```

```
df.shape
```

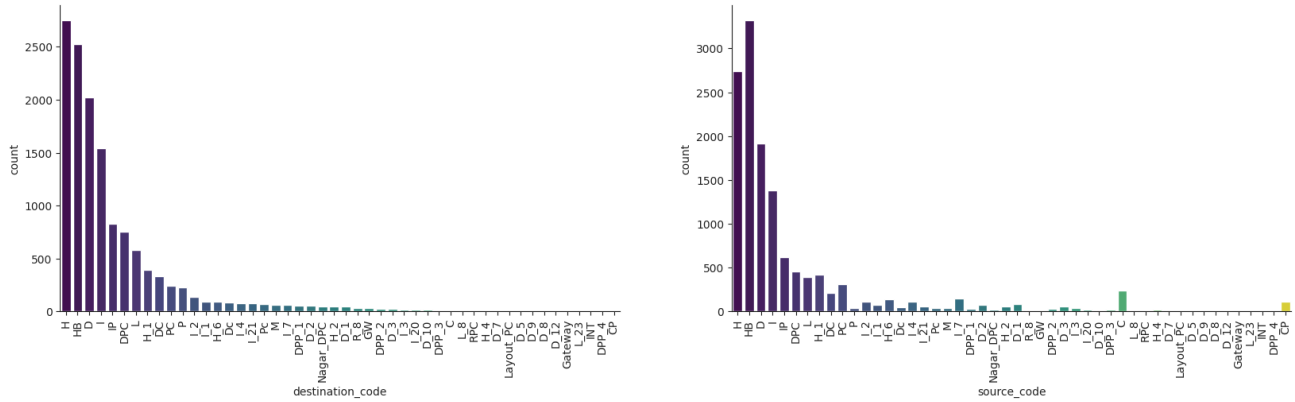
```
(14787, 26)
```

```
df['week'] = df['trip_creation_time'].dt.day_name()
df['hour'] = df['trip_creation_time'].dt.hour
df['date'] = df['trip_creation_time'].dt.date
```

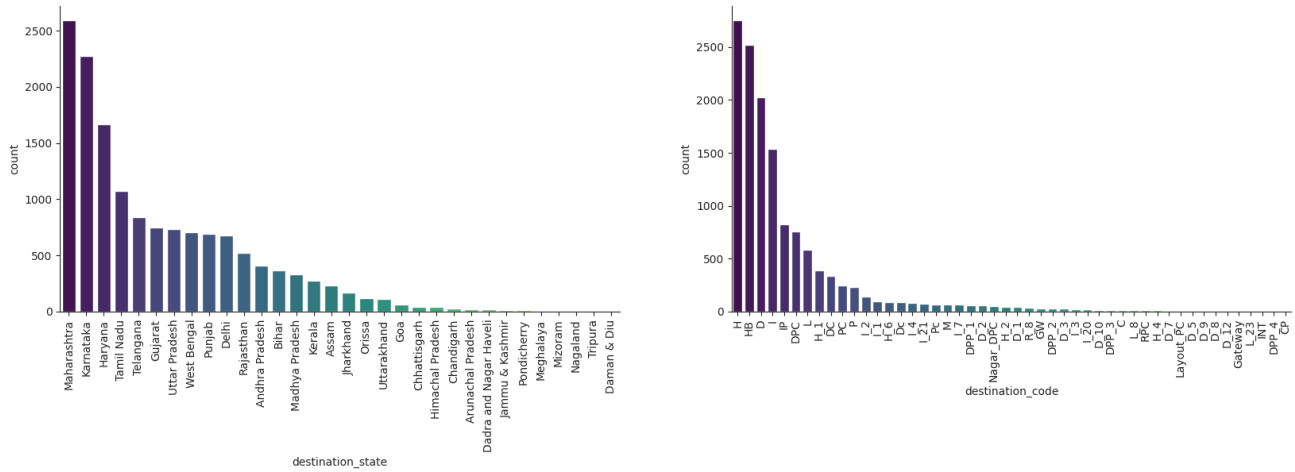
```
plt.figure(figsize=(22,9))
plt.subplot(3,2,1)
sns.countplot(x=df.data,palette='viridis')

plt.subplot(3,2,2)
sns.countplot(x=df.route_type,palette='viridis')

plt.subplot(3,2,3)
sns.countplot(x=df.trip_creation_year,palette='viridis')

plt.subplot(3,2,4)
sns.countplot(x=df.trip_creation_month,palette='viridis')

plt.subplot(3,2,5)
sns.countplot(x=df.week,palette='viridis',order=df.week.value_counts().index,hue=df.route_type)

plt.subplot(3,2,6)
sns.countplot(x=df.hour,palette='viridis',order=df.hour.value_counts().index,hue=df.route_type)
plt.show()
```



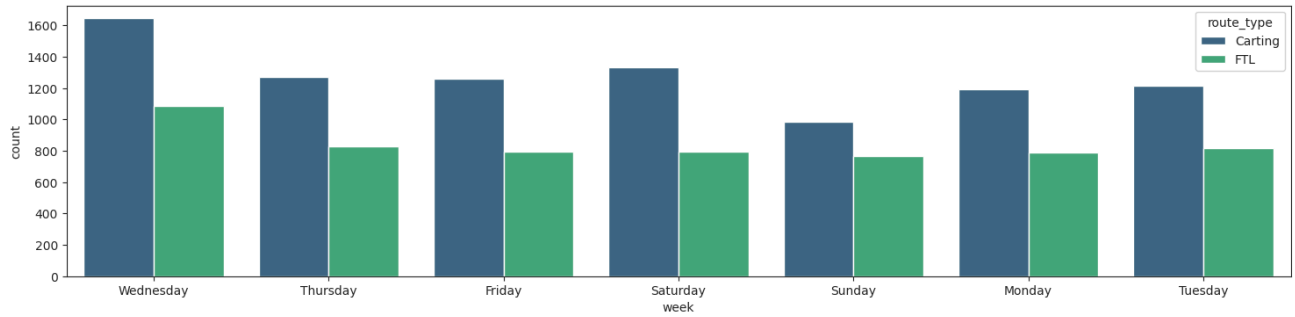

```
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
sns.countplot(x=df.destination_code,palette='viridis',order = df['destination_code'].value_counts().index)
plt.xticks(rotation=90)
plt.subplot(1,2,2)
sns.countplot(x=df.source_code,palette='viridis',order = df['destination_code'].value_counts().index)
plt.xticks(rotation=90)
sns.despine()
plt.show()
```



```
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
sns.countplot(x=df.destination_state,palette='viridis',order = df['destination_state'].value_counts().index)
plt.xticks(rotation=90)
plt.subplot(1,2,2)
sns.countplot(x=df.destination_code,palette='viridis',order = df['destination_code'].value_counts().index)
plt.xticks(rotation=90)
sns.despine()
plt.show()
```



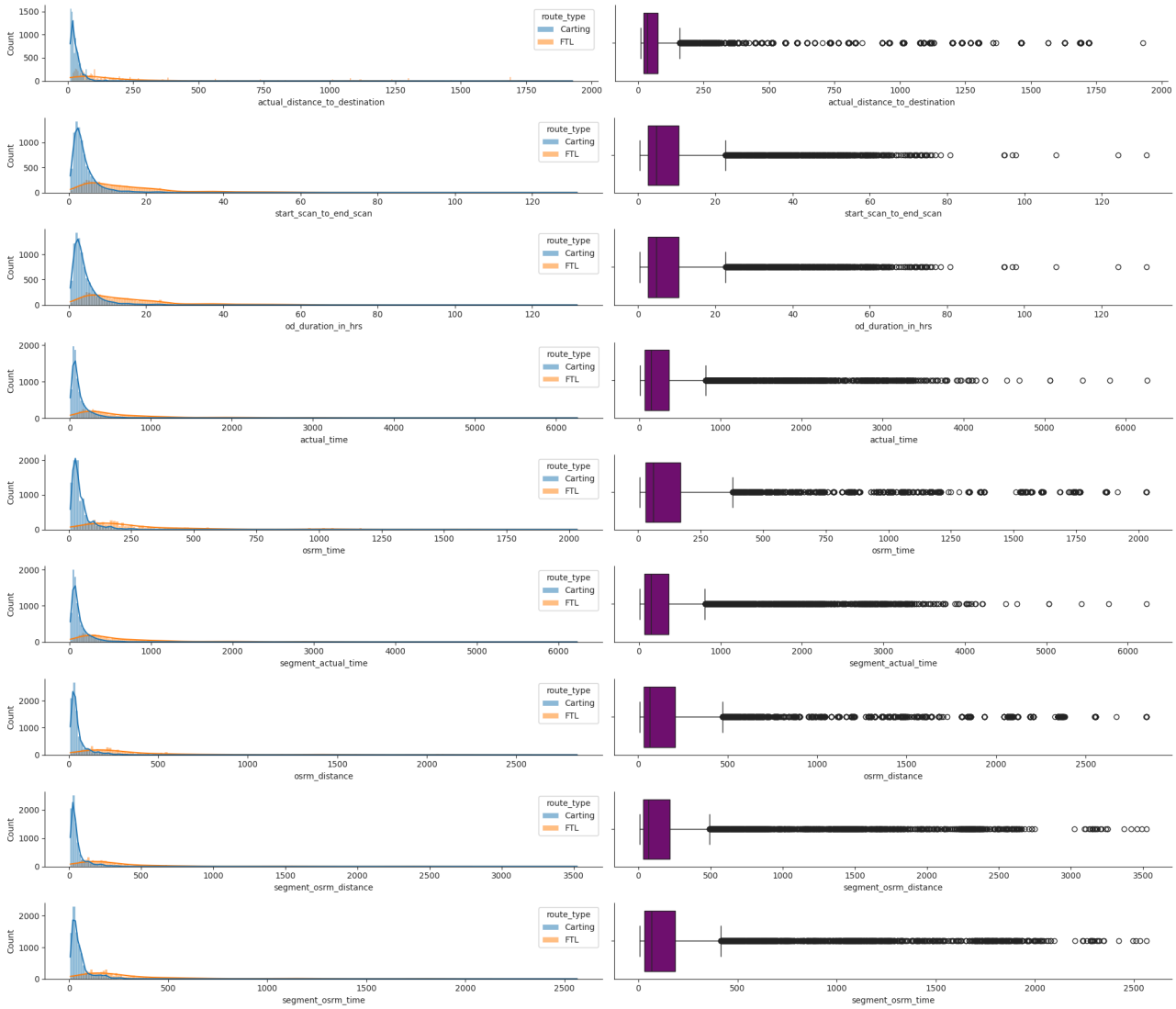

```
plt.figure(figsize=(18,4))
sns.countplot(x=df.week,hue=df.route_type,palette='viridis')
plt.show()
```



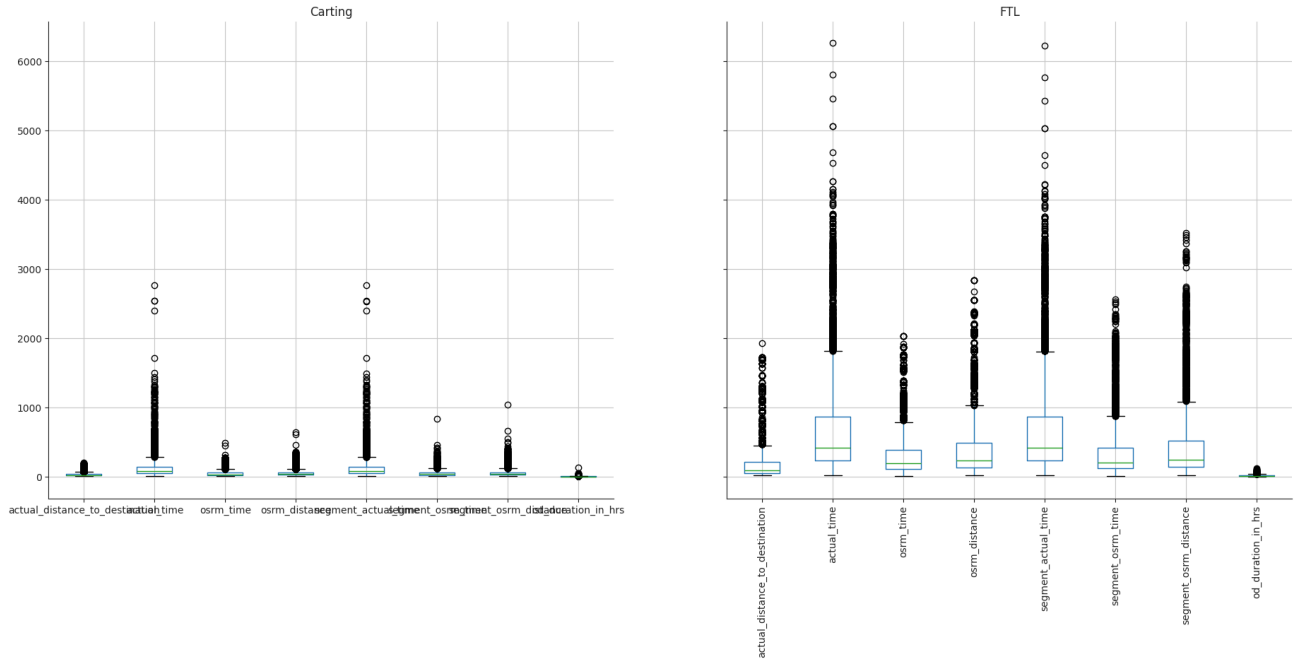

```
number_df = df.select_dtypes('float64')
number_df.shape
```

```
(14787, 9)
```

```
for i in number_df.columns:
    plt.figure(figsize=(20,2))
    plt.subplot(1,2,1)
    sns.histplot(x=number_df[i],kde=True,hue=df.route_type)
    plt.subplot(1,2,2)
    sns.boxplot(x=number_df[i],color='purple')
    sns.despine()
    plt.tight_layout()
    plt.show()
```




```
# plt.figure(figsize=(22,7))
df.groupby('route_type').boxplot(column=['actual_distance_to_destination', 'actual_time', 'osrm_time', 'osrm_distance', 'segmen
       'segment_osrm_time', 'segment_osrm_distance', 'od_duration_in_hrs'],figsize=(22,9))
plt.xticks(rotation=90)
sns.despine()
plt.show()
```



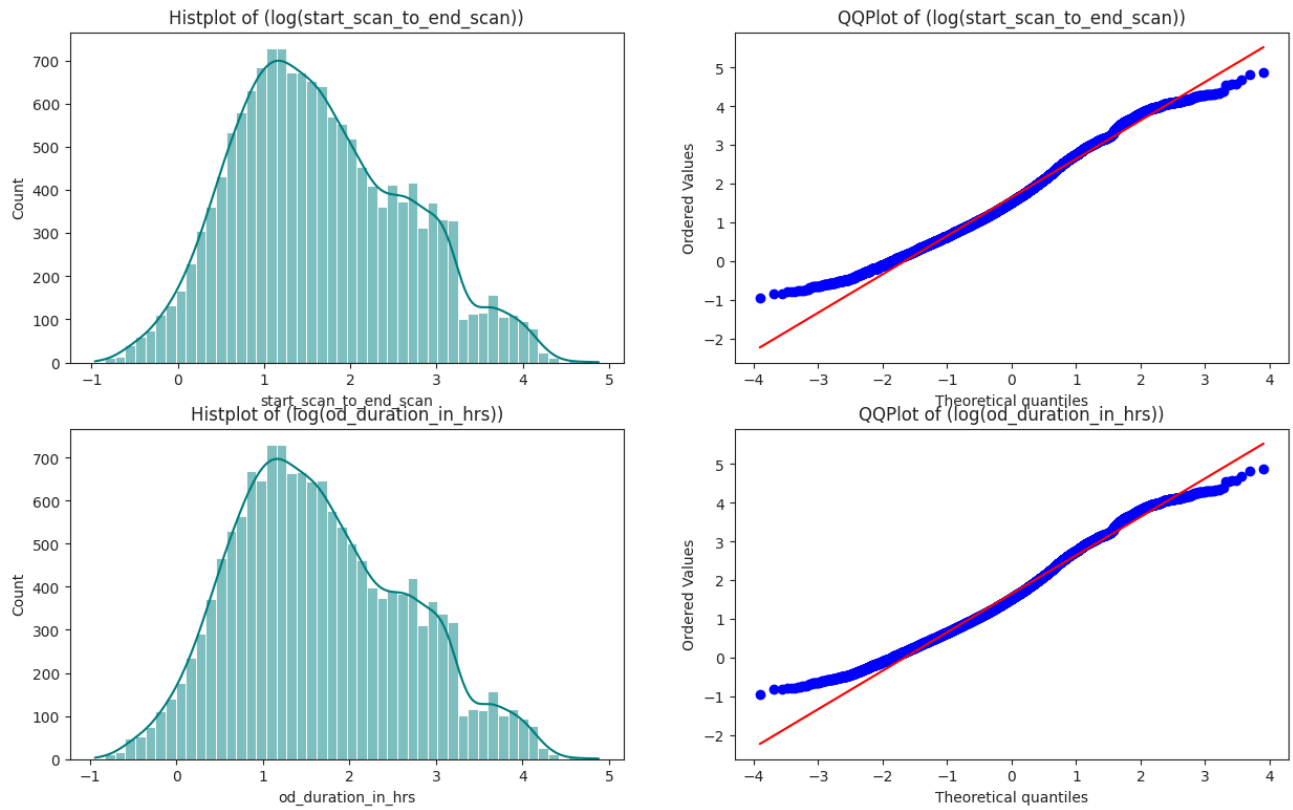
HYPOTHESIS TESTING

1. Statement: Compare the difference between Point a. and start_scan_to_end_scan.

```
from scipy import stats
from scipy.stats import norm
import statsmodels.api as sm
import warnings
warnings.filterwarnings("ignore")
```

```
hypo_1= df.start_scan_to_end_scan
hypo_2 = df.od_duration_in_hrs
hypo_1_1_log = np.log(hypo_1)
hypo_1_2_log = np.log(hypo_2)
```

```
fig = plt.figure(figsize=(15,9))
ax = fig.add_subplot(221)
sns.histplot(hypo_1_1_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(start_scan_to_end_scan))')

ax = fig.add_subplot(222)
stats.probplot(hypo_1_1_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(start_scan_to_end_scan))')

ax = fig.add_subplot(223)
sns.histplot(hypo_1_2_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(od_duration_in_hrs))')

ax = fig.add_subplot(224)
stats.probplot(hypo_1_2_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(od_duration_in_hrs))')
plt.show()
```



Step 1: Defining the null and Altenate Hypothesis

H0: start_scan_to_end_scan and od_duration_in_hrs have the same mean.

H1: start_scan_to_end_scan and od_duration_in_hrs does not have the same mean

Step 2: Selecting the appropriate test

T-test is the appropriate test for this hypothesis because the standard deviation of the population of both the column is unknown.

This is a one-tailed test concerning two population means from two independent populations.

As the population standard deviations are unknown, the two sample independent t-test will be the

appropriate test for this problem.

```
a = df['start_scan_to_end_scan'].sample(1000)
b = df['od_duration_in_hrs'].sample(10000)
print('The standard deviation of the sample of start_scan_to_end_scan is',a.std(),'.')
print('The standard deviation of the sample of od_duration_in_hrs is',b.std(),'.')
```

```
The standard deviation of the sample of start_scan_to_end_scan is 11.624513294203107 .
The standard deviation of the sample of od_duration_in_hrs is 10.993254781360857 .
```

Step 3: Defining the significance level

The significance value, for this hypothesis would be 0.05.

Step 4: Calculating the p_value

```
hypo1 = df.start_scan_to_end_scan
hypo2 = df.od_duration_in_hrs

for i in range(5):
    print(stats.ttest_ind(np.log(hypo1).sample(10000),np.log(hypo2).sample(10000)))
    print()
```

```
TtestResult(statistic=np.float64(-1.123190089448219), pvalue=np.float64(0.26137025432763633), df=np.float64(19998.0))
```

```
TtestResult(statistic=np.float64(-0.8862101025385482), pvalue=np.float64(0.3755149771908376), df=np.float64(19998.0))

TtestResult(statistic=np.float64(0.2934460822067976), pvalue=np.float64(0.7691842514531468), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-0.08789992896366441), pvalue=np.float64(0.9299570931905791), df=np.float64(19998.0))

TtestResult(statistic=np.float64(0.07190210011375633), pvalue=np.float64(0.9426805377623473), df=np.float64(19998.0))
```

Step 5: Conclussion

Multiple t-tests have been done, as observed, p_value > the alpha or significance value.

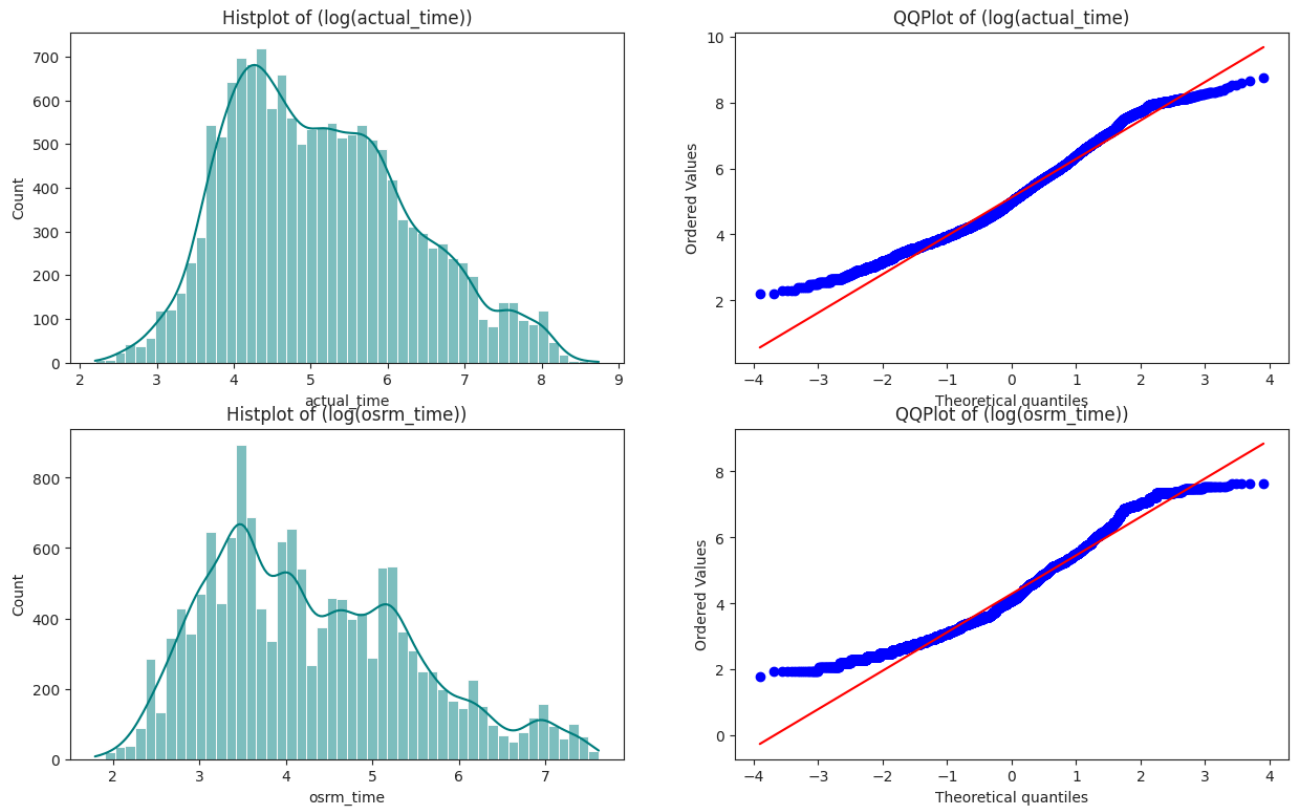
Hence, we fail to reject the null Hypothesis.

Hypothesis 2

Statement: Hypothesis Testing on actual_time aggregated value and OSRM time aggregated value

```
hypo_3 = df.actual_time
hypo_4 = df.osrm_time
hypo_2_1_log = np.log(hypo_3)
hypo_2_2_log = np.log(hypo_4)
```

```
fig = plt.figure(figsize=(15,9))
ax = fig.add_subplot(221)
sns.histplot(hypo_2_1_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(actual_time))')

ax = fig.add_subplot(222)
stats.probplot(hypo_2_1_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(actual_time)')

ax = fig.add_subplot(223)
sns.histplot(hypo_2_2_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(osrm_time))')

ax = fig.add_subplot(224)
stats.probplot(hypo_2_2_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(osrm_time))')
plt.show()
```



Step 1: Defining the null and Altenate Hypothesis

H0: actual_time and osrm_time have the same mean.

H1: actual_time and osrm_time does not have the same mean.

Step 2: Selecting the appropriate test

T-test is the appropriate test for this hypothesis because the standard deviation of the population of both the column is unknown.

This is a one-tailed test concerning two population means from two independent populations.

As the population standard deviations are unknown, the two sample independent t-test will be the

appropriate test for this problem.

Also performing a kruksal test that is a non-parametric method for testing whether samples originate from the same distribution.

```
a = df['actual_time'].sample(1000)
b = df['osrm_time'].sample(10000)
print('The standard deviation of the sample of actual_time sample is',a.std(),'.')
print('The standard deviation of the sample of osrm_time sample is',b.std(),'.')
```

```
The standard deviation of the sample of actual_time sample is 573.1250472835956 .
The standard deviation of the sample of osrm_time sample is 272.1825815374837 .
```

Step 3: Defining the significance level

The significance value, for this hypothesis would be 0.05.

Step 4: Calculating the p_value

```
for i in range(5):
    print(stats.ttest_ind(hypo_2_1_log.sample(5000),hypo_2_2_log.sample(5000)))
    print()
```

```
TtestResult(statistic=np.float64(34.65037763718791), pvalue=np.float64(1.4903949424502402e-248), df=np.float64(9998.0))

TtestResult(statistic=np.float64(34.13294278823976), pvalue=np.float64(1.2302313199989242e-241), df=np.float64(9998.0))
```

```
TtestResult(statistic=np.float64(33.9889067045866), pvalue=np.float64(1.0017237692303823e-239), df=np.float64(9998.0))

TtestResult(statistic=np.float64(34.02829469804291), pvalue=np.float64(3.0120852610547646e-240), df=np.float64(9998.0))

TtestResult(statistic=np.float64(34.90123427159187), pvalue=np.float64(6.17545262736971e-252), df=np.float64(9998.0))
```

Step 5: Conclussion

Multiple t-tests have been done, as observed, p_value > the alpha or significance value.

Hence, we fail to reject the null Hypothesis.

Hypothesis 3

Statement: HypothesisTesting between actual_time aggregated value and segment actual time aggregated value

Step 1: Defining the null and Altenate Hypothesis

H0: actual_time and osrm_time have the same mean.

H1: actual_time and osrm_time does not have the same mean.

Step 2: Selecting the appropriate test

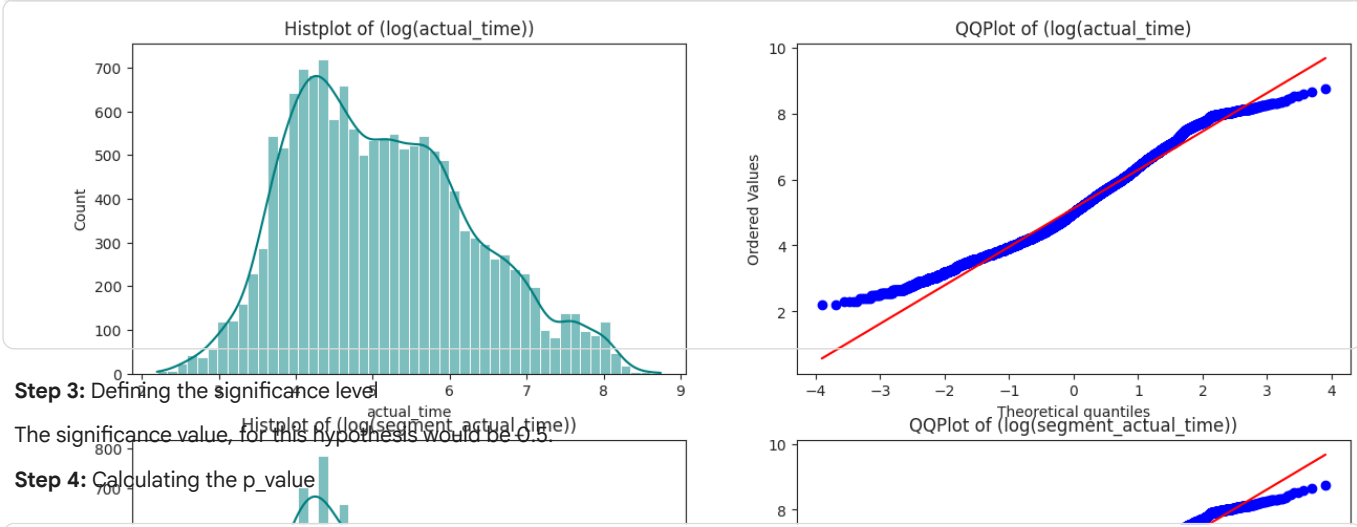
T-test is the appropriate test for this hypothesis because the standard deviation of the population of both the column is unknown.

This is a one-tailed test concerning two population means from two independent populations.

As the population standard deviations are unknown, the two sample independent t-test will be the appropriate test for this problem.

```
hypo_5 = df.actual_time
hypo_6 = df.segment_actual_time
hypo_3_1_log = np.log(hypo_5)
hypo_3_2_log = np.log(hypo_6)
```

```
fig = plt.figure(figsize=(15,9))
ax = fig.add_subplot(221)
sns.histplot(hypo_3_1_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(actual_time))')

ax = fig.add_subplot(222)
stats.probplot(hypo_3_1_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(actual_time)')

ax = fig.add_subplot(223)
sns.histplot(hypo_3_2_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(segment_actual_time))')

ax = fig.add_subplot(224)
stats.probplot(hypo_3_2_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(segment_actual_time))')
plt.show()
```



Step 3: Defining the significance level

The significance value, for this hypothesis would be 0.5.

Step 4: Calculating the p_value

```
for i in range(5):
    print(stats.ttest_ind(hypo_3_1_log.sample(5000),hypo_3_2_log.sample(5000)))
    print()
```

```
TtestResult(statistic=np.float64(0.7810697851923268), pvalue=np.float64(0.4347799450415928), df=np.float64(9998.0))

TtestResult(statistic=np.float64(-0.5705134910533929), pvalue=np.float64(0.5683422954838717), df=np.float64(9998.0))

TtestResult(statistic=np.float64(1.765547077822039), pvalue=np.float64(0.07750238392911134), df=np.float64(9998.0))

TtestResult(statistic=np.float64(0.7160042551810057), pvalue=np.float64(0.4740054436386601), df=np.float64(9998.0))

TtestResult(statistic=np.float64(1.4180135755476262), pvalue=np.float64(0.15621796742585836), df=np.float64(9998.0))
```

Step 5: Conclusion

Multiple t-tests have been done, as observed, p_value > the alpha or significance value.

Hence, we fail to reject the null Hypothesis.

Hypothesis 4

Statement: HypothesisTesting between osrm distance aggregated value and segment osrm distance aggregated value

Step 1: Defining the null and Altenate Hypothesis

H0: actual_time and osrm_time have the same mean.

H1: actual_time and osrm_time does not have the same mean.

Step 2: Selecting the appropriate test

T-test is the appropriate test for this hypothesis because the standard deviation of the population of both the column is unknown.

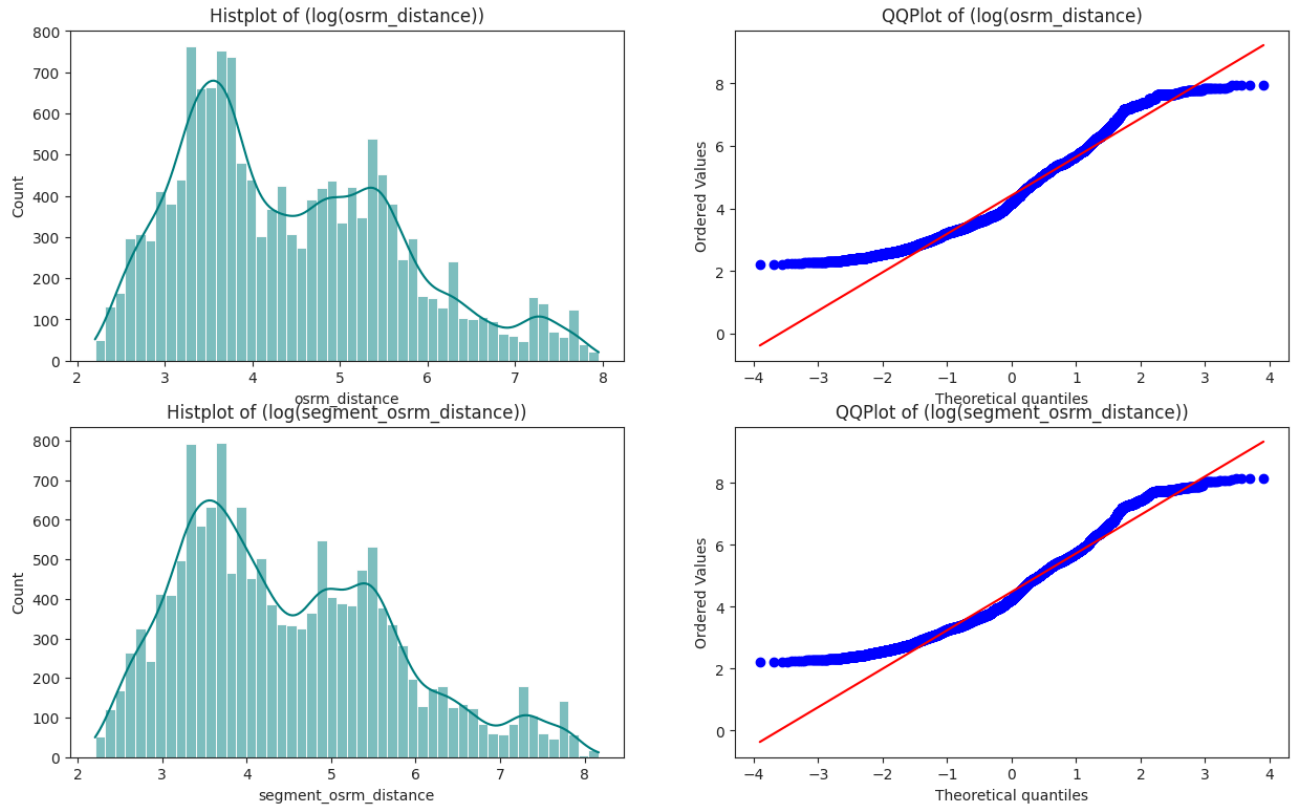
This is a one-tailed test concerning two population means from two independent populations.

As the population standard deviations are unknown, the two sample independent t-test will be the appropriate test for this problem.

```
hypo_7 = df.osrm_distance
hypo_8 = df.segment_osrm_distance
hypo_4_1_log = np.log(hypo_7)
hypo_4_2_log = np.log(hypo_8)
```

```
fig = plt.figure(figsize=(15,9))
ax = fig.add_subplot(221)
sns.histplot(hypo_4_1_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(osrm_distance))')

ax = fig.add_subplot(222)
stats.probplot(hypo_4_1_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(osrm_distance)')

ax = fig.add_subplot(223)
sns.histplot(hypo_4_2_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(segment_osrm_distance))')

ax = fig.add_subplot(224)
```

```
stats.probplot(hypo_4_2_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(segment_osrm_distance))')
plt.show()
```



Step 3: Defining the significance level

The significance value, for this hypothesis would be 0.5.

Step 4: Calculating the p_value

```
for i in range(7):
    print(stats.ttest_ind(hypo_4_1_log.sample(10000) ,hypo_4_2_log.sample(10000)))
    print()
```

```
TtestResult(statistic=np.float64(-2.8449357683432686), pvalue=np.float64(0.0044465491990068485), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-2.423470319537496), pvalue=np.float64(0.01538182363640279), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-3.69993799125338), pvalue=np.float64(0.0002162300348368886), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-4.13118863633427), pvalue=np.float64(3.6235949993312675e-05), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-3.817435579805219), pvalue=np.float64(0.00013525245633729964), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-3.029273944682796), pvalue=np.float64(0.0024545512891278706), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-2.7582164597596814), pvalue=np.float64(0.005817048320656882), df=np.float64(19998.0))
```

Step 5: Conclusion

Multiple t-tests have been done, as observed, p_value < the alpha or significance value.

Hence, we reject the null Hypothesis.

Hypothesis 5

Statement: HypothesisTesting between osrm time aggregated value and segment osrm time aggregated value

Step 1: Defining the null and Altenate Hypothesis

H0: actual_time and osrm_time have the same mean.

H1: actual_time and osrm_time does not have the same mean.

Step 2: Selecting the appropriate test

T-test is the appropriate test for this hypothesis because the standard deviation of the population of both the column is unknown.

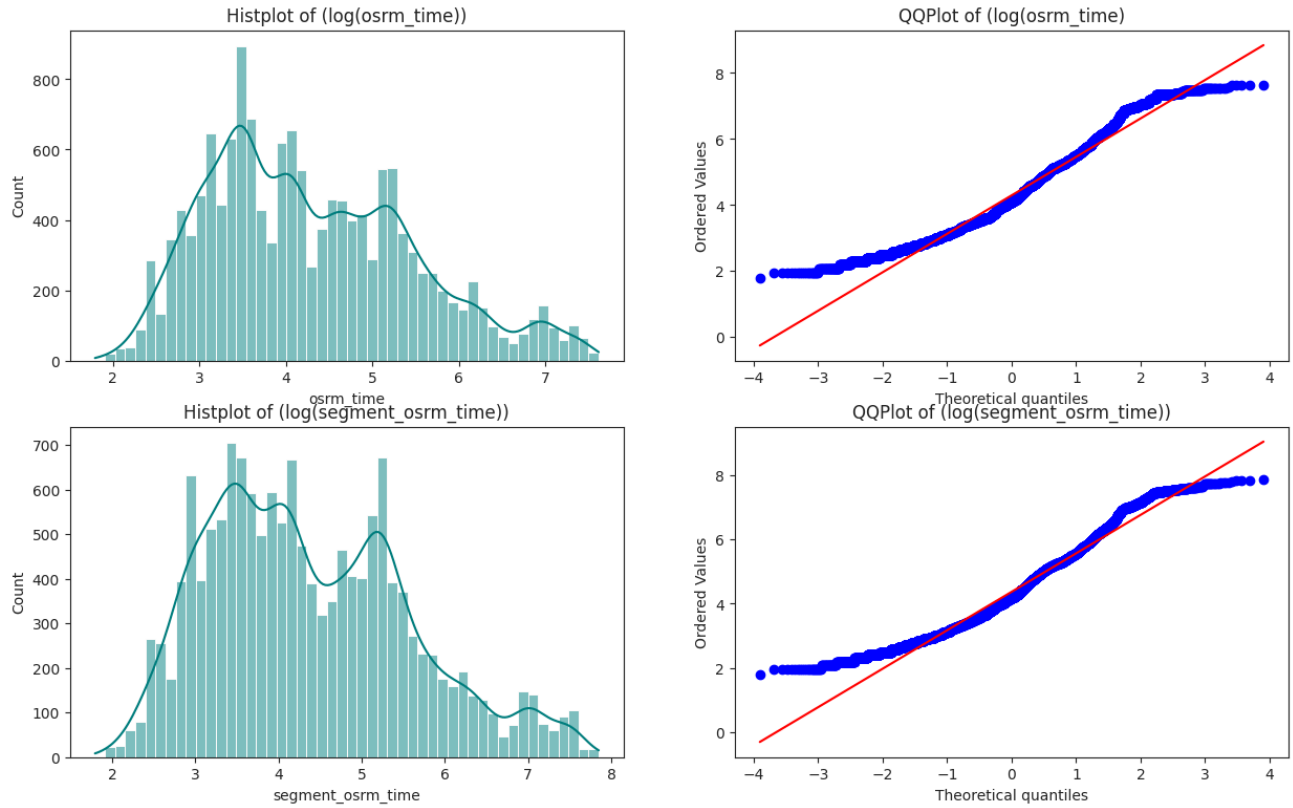
This is a one-tailed test concerning two population means from two independent populations.

As the population standard deviations are unknown, the two sample independent t-test will be the

appropriate test for this problem.

```
hypo_5_1_log = np.log(df.osrm_time)
hypo_5_2_log = np.log(df.segment_osrm_time)
```

```
fig = plt.figure(figsize=(15,9))
ax = fig.add_subplot(221)
sns.histplot(hypo_5_1_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(osrm_time))')

ax = fig.add_subplot(222)
stats.probplot(hypo_5_1_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(osrm_time)')

ax = fig.add_subplot(223)
sns.histplot(hypo_5_2_log, bins=50, kde=True,color='teal')
plt.title('Histplot of (log(segment_osrm_time))')

ax = fig.add_subplot(224)
stats.probplot(hypo_5_2_log, dist='norm', plot=plt)
plt.title('QQPlot of (log(segment_osrm_time))')
plt.show()
```



Step 3: Defining the significance level

The significance value, for this hypothesis would be 0.5.

Step 4: Calculating the p_value

```
for i in range(7):
    print(stats.ttest_ind(hypo_5_1_log.sample(10000) ,hypo_5_2_log.sample(10000)))
    print()
```

```
TtestResult(statistic=np.float64(-4.832761640387538), pvalue=np.float64(1.3565049898389167e-06), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-4.412403416500131), pvalue=np.float64(1.0276374959417271e-05), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-3.7577061156630878), pvalue=np.float64(0.00017196518490562068), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-5.652092752738315), pvalue=np.float64(1.6066562051456484e-08), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-4.5103496630234945), pvalue=np.float64(6.50888371243097e-06), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-4.829583505761545), pvalue=np.float64(1.3783095262515318e-06), df=np.float64(19998.0))

TtestResult(statistic=np.float64(-4.549949908697483), pvalue=np.float64(5.39741802431678e-06), df=np.float64(19998.0))
```

Step 5: Conclusion

Multiple t-tests have been done, as observed, p_value < the alpha or significance value.

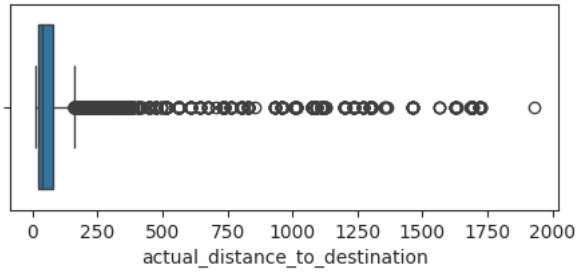
Hence, we reject the null Hypothesis.

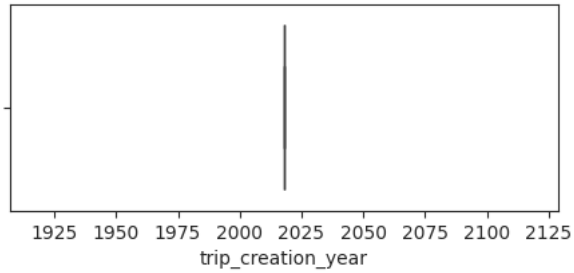
Handle outliers using the IQR method.

```
# Step 1: Identify numerical columns
numerical_cols = df.select_dtypes(include=['number']).columns

# Step 2: Outlier Detection + Handling via IQR
def handle_outliers_iqr(data, cols):
  cleaned_df = data.copy()
  for col in cols:
    Q1 = cleaned_df[col].quantile(0.25)
    Q3 = cleaned_df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    cleaned_df = cleaned_df[(cleaned_df[col] >= lower_bound) & (cleaned_df[col] <= upper_bound)]
  return cleaned_df
```

Visualatisation of outliers before handling them

```
for i, col in enumerate(numerical_cols):
  plt.figure(figsize=(18,14))
  plt.subplot(6,3, i + 1)
  sns.boxplot(x=df[col])

plt.tight_layout()
plt.show()
```

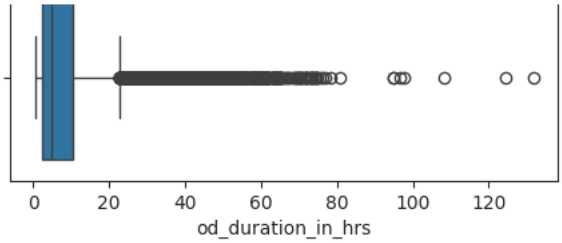


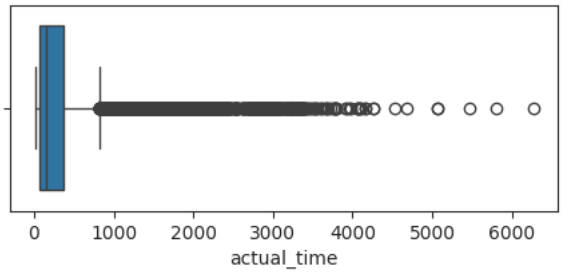


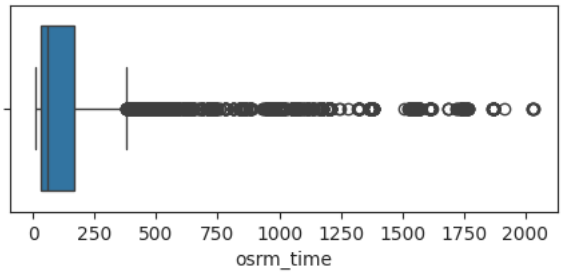
Visualatisation of outliers after handling them

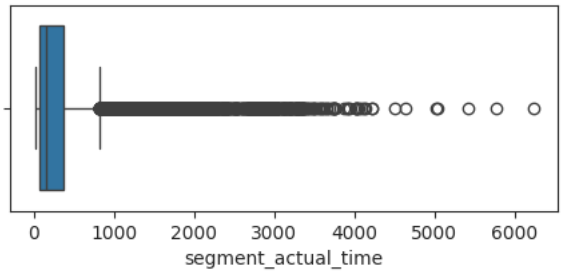
```
df_cleaned = handle_outliers_iqr(df, numerical_cols)
```

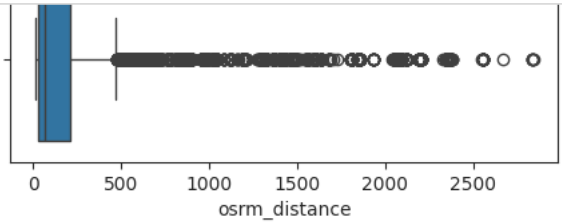
```
for i, col in enumerate(numerical_cols):
  plt.figure(figsize=(18,14))
  plt.subplot(6,3, i + 1)
  sns.boxplot(x=df_cleaned[col])
plt.show()
```

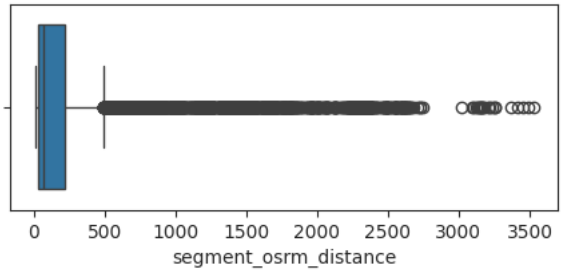


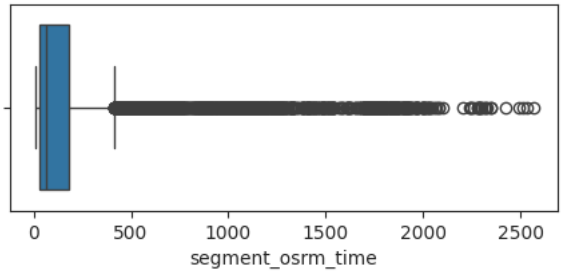


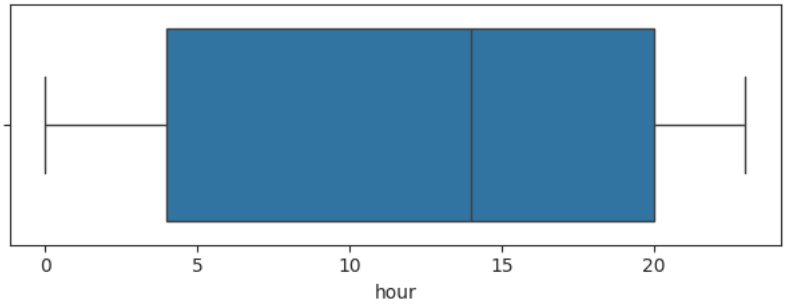


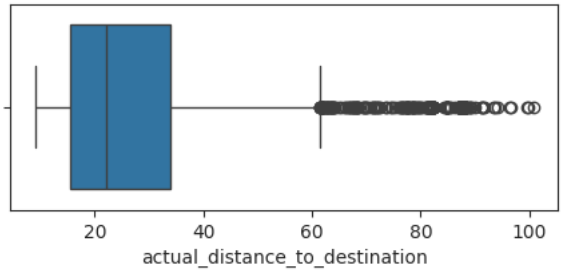


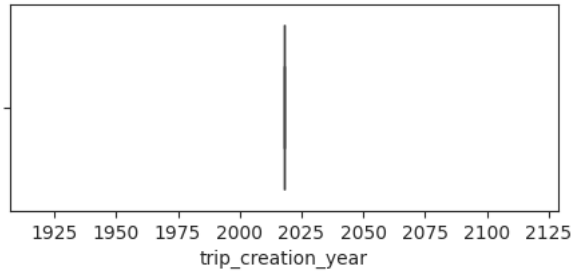

0
500
1000
1500
2000
2500
osrm_distance

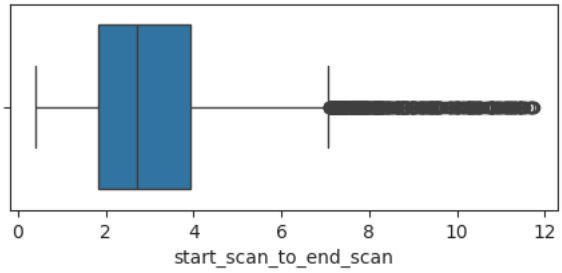

0
500
1000
1500
2000
2500
3000
3500
segment_osrm_distance

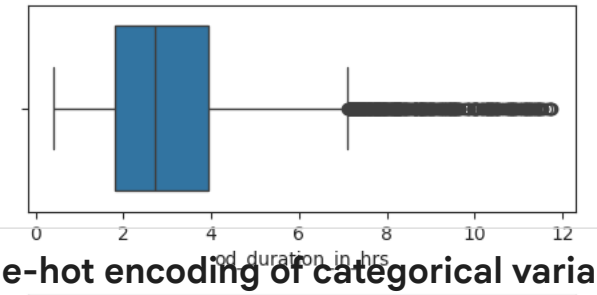

0
500
1000
1500
2000
2500
segment_osrm_time


0
5
10
15
20
hour







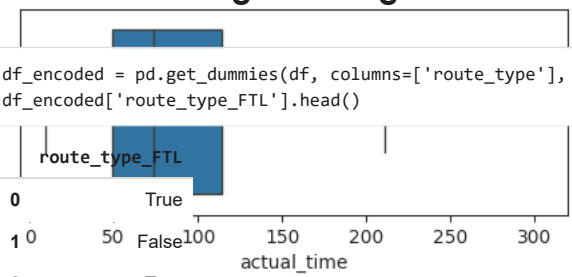


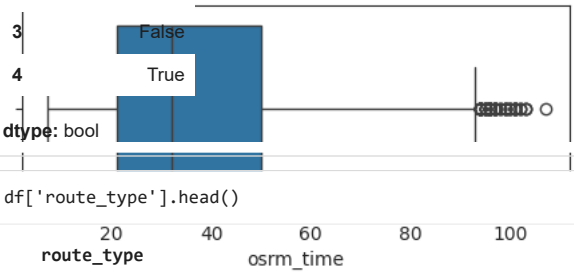
one-hot encoding of categorical variables - (route_type)

```
df_encoded = pd.get_dummies(df, columns=['route_type'], drop_first=True)
df_encoded['route_type_FTL'].head()
```

	route_type_FTL
0	True
1	False
2	True
3	False
4	True

dtype: bool

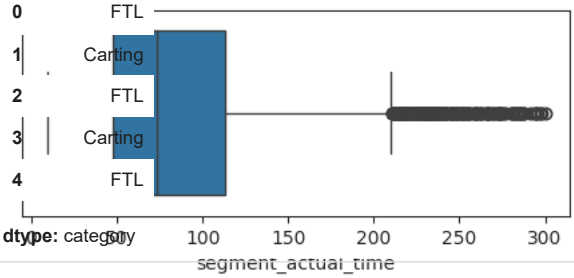





```
df['route_type'].head()
```

	route_type
0	FTL
1	Carting
2	FTL
3	Carting
4	FTL

dtype: category



Normalize/ Standardize the numerical features using MinMaxScaler or StandardScaler

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

numerical_cols = df.select_dtypes(include=['number']).columns
scaler = MinMaxScaler()
df_scaled = df.copy()

df_scaled[numerical_cols] = scaler.fit_transform(df_scaled[numerical_cols])
```

```
df_scaled.head()
```

	trip_uuid	data	trip_creation_time	route_type	route_schedule_uuid	source_center	source_code	source_city
0	trip-153671041653548748	training	2018-09-12 00:00:16.535741	FTL	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	IND462022AAA	H	Bhopal
1	trip-153671042288605164	training	2018-09-12 00:00:22.886430	Carting	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	IND572101AAA	I	Tumkur